

PROJECT REPORT

JAVA TYPING SPEED TEST

A Console-Based Typing Speed Measurement Program

Submitted By

ABPNEET SINGH

B.Tech CSE / AI & ML

Submitted To

MR. SANDEEP MONGA

Department of Computer Science and Engineering

VIT Bhopal University

Academic Year: 2026–27

Acknowledgement

I would like to express my sincere gratitude to my faculty for providing me with the opportunity to develop this project.

The development of this project helped me understand and apply fundamental Java programming concepts such as strings, user input, loops, conditional statements, character comparison, and time measurement.

I would also like to thank my classmates and friends for their support and suggestions during the development and testing of the project.

Abstract

The **Java Typing Speed Test** is a simple console-based application developed using the Java programming language. The purpose of this project is to measure a user's typing speed and accuracy.

The program displays a predefined sentence to the user. After the user presses Enter, a timer starts and the user types the displayed sentence. When the user finishes typing, the timer stops.

The program then calculates the total time taken, typing speed in Words Per Minute (WPM), and typing accuracy by comparing the typed characters with the original sentence.

This project demonstrates the practical use of fundamental Java programming concepts and provides a simple way to evaluate typing performance.

Contents

1	Introduction	5
1.1	Problem Statement	5
1.2	Objectives	5
1.3	Scope of the Project	6
2	System Requirements	7
2.1	Hardware Requirements	7
2.2	Software Requirements	7
2.3	Technologies Used	8
3	System Design	9
3.1	Working Principle	9
3.2	Flowchart	10
4	Implementation	11
4.1	Taking User Input	11
4.2	Starting the Timer	11
4.3	Stopping the Timer	11
4.4	Counting Words	11
4.5	WPM Calculation	12
4.6	Character Comparison	12
4.7	Accuracy Calculation	13
5	Source Code	14
6	Testing and Results	17
6.1	Test Cases	17
6.2	Sample Output	17
7	Advantages and Limitations	19
7.1	Advantages	19
7.2	Limitations	19

8	Future Enhancements	20
9	Conclusion	21

Chapter 1

Introduction

Typing is an important computer skill used in education, programming, office work, communication, and many other activities. Typing speed and accuracy can be improved through regular practice and measurement.

The Java Typing Speed Test is designed as a simple application that allows a user to measure their typing performance.

The application presents a sentence and records the amount of time taken by the user to type it. After the test, the program calculates the typing speed and accuracy.

The project is implemented entirely in Java and operates through the console.

1.1 Problem Statement

Users may find it difficult to manually determine their typing speed and accuracy.

The objective of this project is to develop a Java program that can automatically measure typing time, calculate Words Per Minute, and determine typing accuracy.

1.2 Objectives

The main objectives of the project are:

- To develop a simple typing speed test using Java.
- To measure the time taken to type a sentence.
- To calculate typing speed in WPM.
- To calculate typing accuracy.
- To compare the original sentence with the user's input.
- To demonstrate basic Java programming concepts.

1.3 Scope of the Project

The current version focuses on a single typing test using a predefined sentence.

The project is intended mainly for educational purposes and provides a foundation for developing a more advanced typing application.

Chapter 2

System Requirements

2.1 Hardware Requirements

- Computer or laptop
- Standard keyboard
- Minimum 2 GB RAM
- Basic processor
- Standard display

2.2 Software Requirements

- Windows, Linux, or macOS
- Java Development Kit (JDK) 8 or later
- VS Code, IntelliJ IDEA, Eclipse, or NetBeans
- Command Prompt or Terminal

2.3 Technologies Used

Technology	Purpose
Java	Application development
Scanner	Taking user input
String	Storing and processing text
Random	Not required in current version
System.currentTimeMillis()	Measuring time

Table 2.1: Technologies Used

Chapter 3

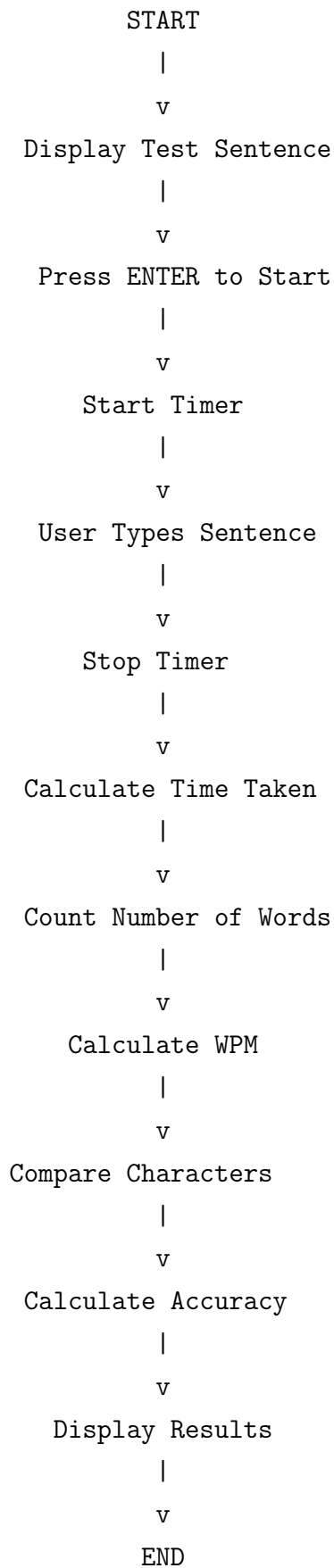
System Design

3.1 Working Principle

The application works according to the following steps:

1. Start the program.
2. Display the typing test heading.
3. Display the sentence.
4. Ask the user to press Enter when ready.
5. Start the timer.
6. Accept the user's typed sentence.
7. Stop the timer.
8. Calculate the time taken.
9. Count the number of words.
10. Calculate WPM.
11. Compare the typed characters with the original sentence.
12. Calculate accuracy.
13. Display the final results.

3.2 Flowchart



Chapter 4

Implementation

4.1 Taking User Input

The Java `Scanner` class is used to take input from the user.

```
1 Scanner sc = new Scanner(System.in);  
2  
3 String typedText = sc.nextLine();
```

4.2 Starting the Timer

The program uses `System.currentTimeMillis()` to record the starting time.

```
1 long startTime = System.currentTimeMillis();
```

4.3 Stopping the Timer

After the user finishes typing, the ending time is recorded.

```
1 long endTime = System.currentTimeMillis();
```

The difference between the starting and ending time gives the total time taken.

```
1 double timeTaken =  
2     (endTime - startTime) / 1000.0;
```

The division by 1000 converts milliseconds into seconds.

4.4 Counting Words

The number of words typed by the user is calculated using the `split()` method.

```
1 int words = typedText.trim().isEmpty()  
2     ? 0  
3     : typedText.trim().split("\\s+").length;
```

4.5 WPM Calculation

Words Per Minute (WPM) is calculated using:

$$WPM = \frac{\text{Number of Words}}{\text{Time in Seconds}} \times 60$$

For example, if a user types 20 words in 30 seconds:

$$WPM = \frac{20}{30} \times 60$$

$$WPM = 40$$

Therefore, the typing speed is 40 WPM.

4.6 Character Comparison

The program compares each character of the original sentence with the corresponding character entered by the user.

```
1 int correctCharacters = 0;  
2  
3 int minLength =  
4     Math.min(sentence.length(),  
5         typedText.length());  
6  
7 for (int i = 0; i < minLength; i++) {  
8  
9     if (sentence.charAt(i) ==  
10         typedText.charAt(i)) {  
11  
12         correctCharacters++;  
13     }  
14 }
```

4.7 Accuracy Calculation

Accuracy is calculated using:

$$Accuracy = \frac{\text{Correct Characters}}{\text{Original Sentence Characters}} \times 100$$

For example, if 40 out of 45 characters are correct:

$$Accuracy = \frac{40}{45} \times 100$$

$$Accuracy = 88.89\%$$

Chapter 5

Source Code

The complete Java source code used in this project is given below.

```
1 import java.util.Scanner;
2
3 public class TypingSpeedTest {
4
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8
9         String sentence =
10             "Java is a powerful programming language.";
11
12         System.out.println("==== TYPING SPEED TEST =====");
13         System.out.println();
14
15         System.out.println(
16             "Type the following sentence:");
17
18         System.out.println();
19         System.out.println(sentence);
20         System.out.println();
21
22         System.out.println(
23             "Press ENTER when you are ready...");
24
25         sc.nextLine();
26
27         // Start timer
28         long startTime =
```

```
29         System.currentTimeMillis();
30
31     System.out.println("Start typing:");
32
33     String typedText =
34         sc.nextLine();
35
36     // Stop timer
37     long endTime =
38         System.currentTimeMillis();
39
40     // Calculate time in seconds
41     double timeTaken =
42         (endTime - startTime) / 1000.0;
43
44     // Calculate number of words
45     int words =
46         typedText.trim().isEmpty()
47         ? 0
48         : typedText.trim()
49           .split("\\s+").length;
50
51     // Calculate WPM
52     double wpm =
53         (words / timeTaken) * 60;
54
55     // Calculate correct characters
56     int correctCharacters = 0;
57
58     int minLength =
59         Math.min(sentence.length(),
60                 typedText.length());
61
62     for (int i = 0;
63         i < minLength;
64         i++) {
65
66         if (sentence.charAt(i) ==
67             typedText.charAt(i)) {
68
69             correctCharacters++;
```

```
70         }
71     }
72
73     // Calculate accuracy
74     double accuracy =
75         ((double) correctCharacters /
76          sentence.length()) * 100;
77
78     // Display results
79     System.out.println();
80     System.out.println("==== RESULTS ====");
81
82     System.out.printf(
83         "Time Taken : %.2f seconds%n",
84         timeTaken);
85
86     System.out.printf(
87         "Typing Speed: %.2f WPM%n",
88         wpm);
89
90     System.out.printf(
91         "Accuracy      : %.2f%%\n",
92         accuracy);
93
94     sc.close();
95 }
96 }
```

Chapter 6

Testing and Results

The program was tested using different typing inputs to verify the correctness of the timer, WPM calculation, and accuracy calculation.

6.1 Test Cases

Test	Condition	Expected Result
1	Correct sentence	High accuracy
2	Typing errors	Lower accuracy
3	Fast typing	Higher WPM
4	Slow typing	Lower WPM
5	Empty input	Program handles input

Table 6.1: Test Cases

6.2 Sample Output

A sample execution of the program is shown below:

```
===== TYPING SPEED TEST =====
```

Type the following sentence:

Java is a powerful programming language.

Press ENTER when you are ready...

Start typing:

Java is a powerful programming language.

===== RESULTS =====

Time Taken : 8.42 seconds

Typing Speed: 42.76 WPM

Accuracy : 100.00%

The output shows three main results:

- Time Taken
- Typing Speed in WPM
- Typing Accuracy

Chapter 7

Advantages and Limitations

7.1 Advantages

- Simple and easy to use.
- Beginner-friendly Java project.
- Requires very few system resources.
- Automatically measures typing time.
- Calculates typing speed.
- Calculates typing accuracy.
- Demonstrates practical Java programming concepts.

7.2 Limitations

- Only one predefined sentence is used.
- The application is console-based.
- There is only one test round.
- Results are not stored permanently.
- There is no graphical user interface.
- There is no leaderboard.

Chapter 8

Future Enhancements

The project can be improved by adding the following features:

- 30-second and 60-second typing modes.
- Multiple random sentences.
- Graphical User Interface using Java Swing or JavaFX.
- Leaderboard and high-score system.
- User profiles.
- Score history.
- Different difficulty levels.
- Performance graphs.
- Database integration.

Chapter 9

Conclusion

The **Java Typing Speed Test** successfully demonstrates the development of a simple interactive application using Java.

The program measures the time taken by a user to type a predefined sentence and uses this information to calculate typing speed in Words Per Minute. It also compares the user's input with the original sentence to calculate typing accuracy.

The project demonstrates several fundamental Java concepts including strings, user input, loops, conditional statements, character comparison, mathematical calculations, and time measurement.

This project provides a foundation for developing a more advanced typing application with multiple test modes, random sentences, graphical interfaces, score storage, and performance tracking.

References

1. Oracle Java Documentation.
2. Java Programming Language Documentation.
3. Herbert Schildt, *Java: The Complete Reference*.
4. Java Standard Library Documentation.